Visitor Pattern

Файловая система в памяти
Java 17 / Maven / JUnit 5

1. Назначение и модель

Программа хранит иерархию файлов и каталогов в памяти. К одному дереву применяются три независимые операции: подсчёт общего размера, поиск файлов и сбор статистики. Работа с диском не требуется.

Element задаёт имя, список детей и метод accept(Visitor). **FileSystemNode** хранит общее имя и закрытую ссылку на родителя. **FileNode** хранит размер в long. **DirectoryNode** управляет дочерними элементами. У файла список детей пустой, поэтому обходчику не требуется проверять тип узла.

2. Двойная диспетчеризация

Первый шаг: element.accept(visitor) выбирает реализацию элемента во время выполнения. Второй шаг: внутри FileNode вызов visitor.visit(this) выбирает перегрузку visit(FileNode), а внутри DirectoryNode - visit(DirectoryNode). Затем вызывается реализация конкретного посетителя. Операции не используют проверки типа, приведения или отражение.

3. Владелец обхода и состояние

FileTreeWalker.walk выполняет обход в глубину, начиная с выбранного корня. Перед запуском он вызывает reset(), а перед accept() передаёт относительный путь и глубину через setContext(). Только обходчик посещает дочерние узлы. Каждый достижимый узел обрабатывается ровно один раз за операцию.

Посетители повторно используются через walk(): предыдущие результаты очищаются. Порядок разных операций не влияет на ответ. Прямой accept() обрабатывает один узел, без обхода и сброса. Обход последовательный; изменение дерева во время обхода не поддерживается.

4. Целостность дерева

Конструкторы запрещают отрицательный размер и недопустимые имена. DirectoryNode.add проверяет null, циклы и одинаковые имена соседей; attachTo запрещает второго родителя. Проверка цикла проходит от текущего каталога по его родителям. Ошибка добавления оставляет дерево без изменений. Наружу возвращается неизменяемая копия списка детей.

Операции и пример

5. Три посетителя

TotalSizeVisitor складывает размеры файлов; каталоги ничего не добавляют. Math.addExact обнаруживает переполнение long вместо получения неверного итога.

SearchVisitor принимает Predicate<String> для имени и минимальный размер. Условия объединяются через AND; граница размера включена. Поиск возвращает неизменяемый список путей, отсортированный лексикографически.

StatisticsVisitor считает файлы, каталоги и максимальную глубину всех узлов, включая пустые каталоги. Корень имеет глубину 0; корневой каталог входит в число каталогов. Путь строится из имён детей через /. Имя выбранного корневого каталога опускается. Если корень - файл, его путь равен имени файла.

6. Данные и фактический вывод Main

```
workspace/
  readme.txt (120)
  documents/
    notes.txt (80), report.txt (450)
    archive/old.txt (100)
  images/logo.png (2048)
  empty/
```

Критерий поиска: имя заканчивается на .txt, размер не меньше 100 байт.

```
Total size: 2798 bytes
Text files with size >= 100 bytes:
  documents/archive/old.txt
  documents/report.txt
  readme.txt
Files: 5
Directories: 5
Maximum depth: 3
```

7. Пять практик Clean Code

Понятные имена: minimumSize и getMaximumDepth прямо описывают ограничение и результат.

Одна ответственность: FileTreeWalker отвечает за обход, SearchVisitor - за отбор файлов.

Малые методы: FileNode.accept содержит один вызов; логика выбора операции видна сразу.

Инкапсуляция: DirectoryNode.getChildren возвращает List.copyOf, поэтому изменения проходят через проверяемый add.

Нет цепочек проверок типа: FileNode.accept и DirectoryNode.accept вызывают перегруженный visit(this); поведение выбирается полиморфно.

Проверка и расширение

8. Автоматические тесты

Выполнено на Java 17 командой mvn test из корня проекта. Результат: **25 тестов, 0 ошибок, 0 неудачных, 0 пропущенных. BUILD SUCCESS.**

Класс тестов	Кол-во	Проверяемое поведение
DirectoryNodeTest	7	Размеры, имена, дубликаты, родители, циклы, закрытые коллекции
FileTreeWalkerTest	2	Однократный обход, пути, глубина, сброс и null
StatisticsVisitorTest	5	Подсчёт, корни, пустые каталоги, глубина и повторный запуск
VisitorDispatchTest	1	Обе перегрузки visit через Element.accept
VisitorOperationsTest	10	Сумма, оба фильтра, сортировка, поддерево, повторы, переполнение

T1 покрыт VisitorDispatchTest; T2-T4 - VisitorOperationsTest; T5 - StatisticsVisitorTest; T6-T7 - VisitorOperationsTest и StatisticsVisitorTest; T8 - DirectoryNodeTest. FileTreeWalkerTest отдельно проверяет передачу контекста и ровно один вызов для каждого элемента.

9. Граница расширения и компромиссы

Первый коммит содержит элементы, интерфейс Visitor, обходчик и две операции. Следующий коммит добавляет StatisticsVisitor, его тесты и вызов в Main. FileNode, DirectoryNode, Visitor и FileTreeWalker при этом не изменяются: перегрузки для обоих типов и методы жизненного цикла уже входят в исходный контракт.

Новая операция добавляется отдельным посетителем. Новый тип элемента обходится дороже: потребуются новая перегрузка Visitor.visit и изменения всех существующих посетителей. Это основной компромисс Visitor при расширении набора элементов.

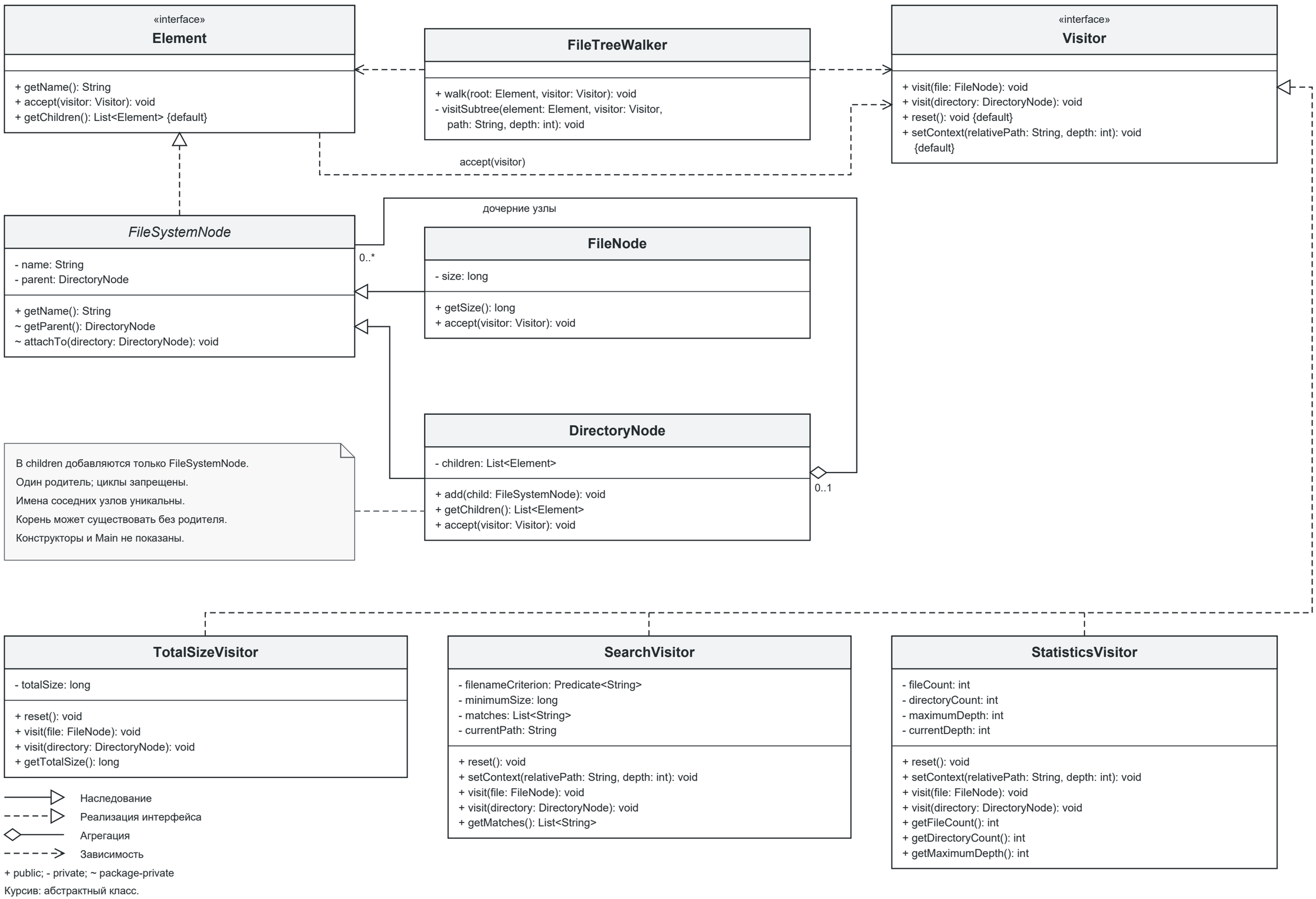
Единый обходчик исключает повторение обхода в операциях, но связывает их с методами reset и setContext. Рекурсивный обход прост для чтения; очень глубокое дерево ограничено стеком Java. Неизменяемые снимки списков защищают состояние ценой копирования.

10. Запуск и репозиторий

```
mvn test
mvn compile
java -cp target/classes kz.aitu.visitor.Main
```

GitHub: anuarmaksut711-dotcom/assignment-6-visitor-pattern

UML-диаграмма классов: Visitor Pattern



Обозначения: Lecture 3, с. 13-14; Lecture 4, с. 9-10.